PROJECT_REPORT.md

Mini-React: A Lightweight Reactive UI Framework

1. Project Overview
| GitHub Repository | [MSIS207.R11.CTTT-static_web-v3r](https://github.com/V3rm0u1h/MSIS207.R11.CTTT-static_web-v3r.git) |
| ChatGPT Conversation 1 | [View chat](hhttps://chatgpt.com/share/6aca6d1c-4c10-83ec-85ed-8961fb4b8667)
1.1 Objective

The objective of this project is to build a lightweight UI framework from scratch using vanilla JavaScript. The project explores how core frontend framework concepts work internally, including Virtual DOM creation, DOM rendering, state management, event delegation, and asynchronous UI updates.

Rather than relying on React or another UI framework, Mini-React implements its own foundational mechanisms and uses them to build reactive user interfaces.

1.2 Project Scope

The project includes the following features:

- Virtual DOM node creation through `createElement()` and `createTextElement()`.
- Recursive rendering of VNodes into real DOM nodes through `renderToDOM()`.
- State management and reactive state updates.
- Root-level event delegation.
- A reactive Task Manager application.
- A resilient asynchronous data component with loading, success, error, and retry behavior.
- Automated testing, accessibility checks, and security verification.

1.3 Technologies Used

- HTML5
- CSS3
- JavaScript ES Modules
- Node.js for automated tests
- Git and GitHub for version control
- Chrome DevTools for DOM inspection and browser verification

2. System Architecture

Mini-React is organized into separate modules, each responsible for a specific part of the UI lifecycle.

Module	Responsibility
`createElement.js`	Creates structured Virtual DOM nodes.

`renderToDOM.js`	Converts VNodes into real DOM elements and text nodes.
`state.js`	Provides the state storage and update mechanism.
`useState.js`	Provides reactive state access where supported by the existing state engine.
`events.js`	Manages delegated event handling.
`app.js`	Implements the Task Manager application.
`asyncDataComponent.js`	Implements the asynchronous data state machine, request handling, and UI.
`asyncDataComponent.css`	Defines the skeleton animation and component styling.

The asynchronous component reuses the existing Virtual DOM factory, renderer, and event-handling architecture instead of introducing another rendering system.

Figure 1 — Project architecture



3. Implementation Milestones

3.1 Virtual DOM Factory

The `createElement()` factory converts element descriptions into a consistent VNode structure containing a type, props object, and children array.

Text values are converted into text VNodes. Nested child arrays are flattened, while valid VNodes are preserved. Null, undefined, and boolean children are ignored according to the factory contract.

The factory creates data structures only; it does not directly create browser DOM nodes.

3.2 Recursive DOM Renderer

The `renderToDOM()` function converts the VNode tree into real DOM nodes.

The renderer:
- Validates the VNode structure.
- Creates text nodes using `document.createTextNode()`.
- Creates semantic elements using `document.createElement()`.
- Applies explicitly supported properties and attributes.
- Recursively renders children in the correct order.
- Preserves the original VNode tree.
- Supports integration with the existing event delegation mechanism.

3.3 State Management and Event Delegation

The state engine provides the foundation for reactive UI updates. The event delegation module allows supported event handlers to be managed

through the existing event architecture instead of attaching separate native listeners to every child.

These mechanisms are reused by the application and the asynchronous data component where their established contracts permit.

3.4 Reactive Task Manager

The Task Manager demonstrates how the framework's state and event mechanisms can support a functional user interface.

The application supports task creation, completion, deletion, and filtering by All, Active, and Completed status. The interface updates in response to user interactions.

3.5 Resilient Asynchronous Data Component

The asynchronous component manages data requests using exactly four UI states:

- `IDLE`: The component is ready to request data.
- `LOADING`: The request is in progress and a skeleton is displayed.
- `SUCCESS`: Validated response data is displayed.
- `ERROR`: A human-readable error message and retry action are displayed.

A central transition mechanism validates state changes. Each request receives a monotonically increasing identifier so that outdated responses cannot overwrite the result of a newer request.

The component also supports request cancellation where possible and invalidates outstanding work during disposal. Cancellation is not relied upon as the only protection against stale responses.

The response is validated before its fields are used. Empty valid responses are handled separately from malformed data.

4. Security and Safe Rendering

Security is an important part of the Mini-React implementation.

4.1 Safe Text Rendering

Text values are rendered through text nodes instead of being parsed as HTML. The implementation does not use `innerHTML` to render untrusted response content.

For the security test, use the following literal string:

`<script>alert(1)</script>`

Expected result:

- The string is displayed literally.
- No script element is created from the string.
- The JavaScript contained in the string does not execute.

4.2 Input and Response Validation

The VNode factory and renderer validate their inputs before creating DOM nodes. The asynchronous component validates response data before using its fields.

The component also avoids exposing stack traces or sensitive implementation details in user-facing error messages.

5. Automated Testing

Automated tests are used to verify the Virtual DOM factory, renderer, and asynchronous component independently.

5.1 Virtual DOM Factory Tests

The factory tests cover:
- VNode structure and nested hierarchy.
- Props and children normalization.
- Recursive child-array flattening.
- Numeric zero and empty-string handling.
- Invalid element types and malformed children.
- Literal handling of HTML-like text.

Recorded result: **18/18 tests passed.**

5.2 Renderer Tests

The renderer tests cover:
- Recursive element creation.
- Text-node rendering.
- Supported attributes and boolean properties.
- Invalid VNode rejection.
- Safe text rendering.
- Root mounting and event compatibility.

Recorded result: **12/12 tests passed.**

5.3 Asynchronous Component Tests

The asynchronous component tests cover:
- Valid and invalid state transitions.
- Successful, failed, and empty responses.
- Stale-response protection.
- Stale rejection handling.
- Retry recovery.
- Cleanup and disposal.
- Safe handling of HTML-like strings.

Recorded result: **10/10 tests passed.**

5.4 Test Execution

Run the following commands from the project root:

```bash
node --experimental-default-type=module createElement.test.js

node --experimental-default-type=module renderToDOM.test.js

node --experimental-default-type=module asyncDataComponent.test.js
```

The recorded combined result is **40/40 automated tests passed**.

6. Accessibility

The implementation considers accessibility in both the semantic DOM structure and the asynchronous component.

The component provides:
- Semantic HTML elements.
- A status region for loading feedback.
- An alert region for errors.
- A clearly named, keyboard-accessible retry button.
- Visible keyboard focus indicators.
- Decorative skeleton elements hidden from assistive technology.
- Reduced-motion support for users who prefer less animation.

7. Performance and Reliability

The component uses CSS for skeleton animation rather than JavaScript animation loops. The loading layout reserves space to reduce layout shifts.

Request identifiers protect the interface from out-of-order asynchronous responses. Request cancellation is used where available, while request identity remains the correctness mechanism.

The component does not introduce polling or require a backend for its demonstration. A mock data source can be used to test success, failure, and retry behavior deterministically.

Performance claims should be supported by measurements if numerical targets are reported.

8. Git and Version Control

The project is organized into implementation milestones to keep the work reviewable and reduce unrelated changes within commits.

The asynchronous component milestone uses the required commit message:

```text
feat(ui): implement multi-state data component with skeleton feedback
```

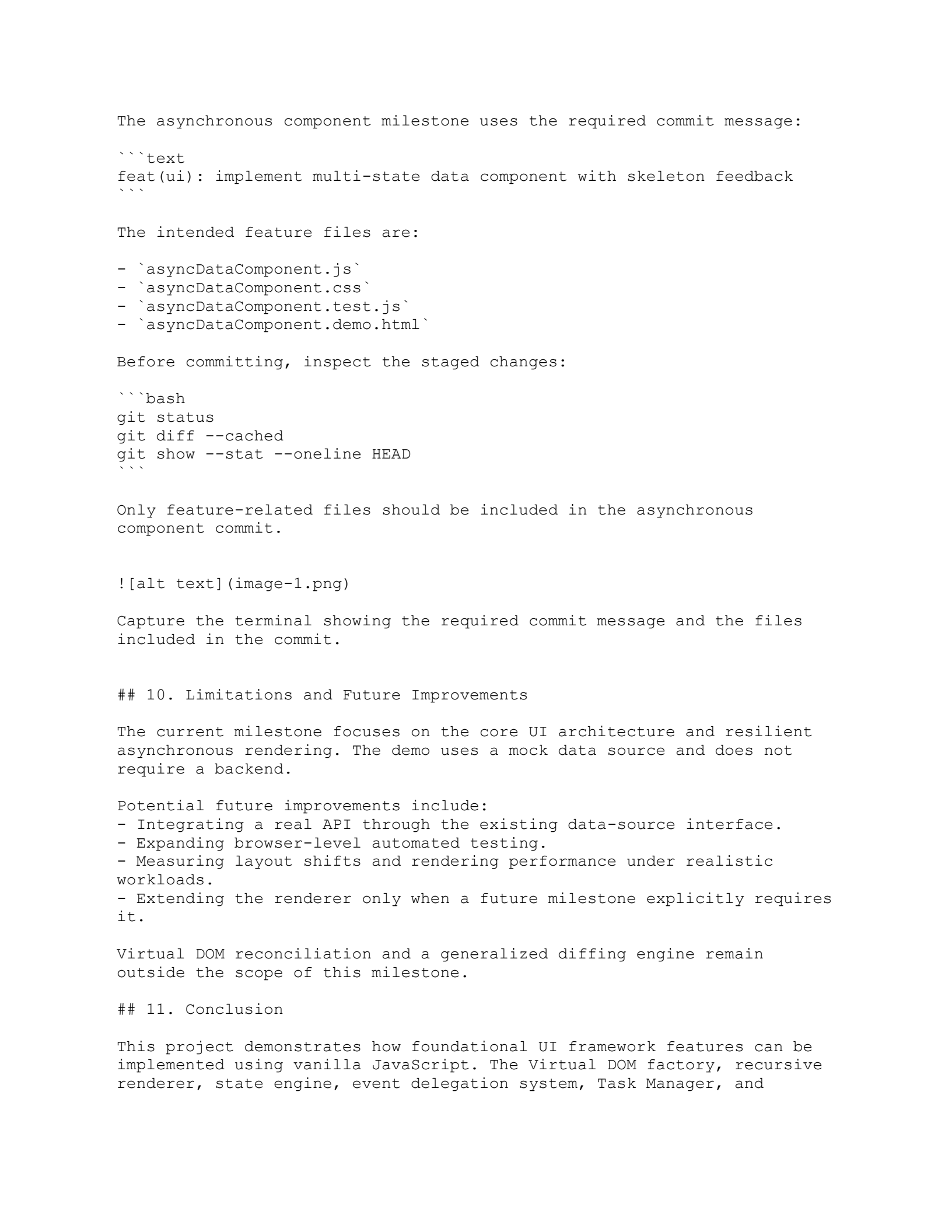
The intended feature files are:

- `asyncDataComponent.js`
- `asyncDataComponent.css`
- `asyncDataComponent.test.js`
- `asyncDataComponent.demo.html`

Before committing, inspect the staged changes:

```bash
git status
git diff --cached
git show --stat --oneline HEAD
```

Only feature-related files should be included in the asynchronous component commit.



Capture the terminal showing the required commit message and the files included in the commit.

10. Limitations and Future Improvements

The current milestone focuses on the core UI architecture and resilient asynchronous rendering. The demo uses a mock data source and does not require a backend.

Potential future improvements include:
- Integrating a real API through the existing data-source interface.
- Expanding browser-level automated testing.
- Measuring layout shifts and rendering performance under realistic workloads.
- Extending the renderer only when a future milestone explicitly requires it.

Virtual DOM reconciliation and a generalized diffing engine remain outside the scope of this milestone.

11. Conclusion

This project demonstrates how foundational UI framework features can be implemented using vanilla JavaScript. The Virtual DOM factory, recursive renderer, state engine, event delegation system, Task Manager, and

resilient asynchronous data component provide a modular foundation for building reactive user interfaces.

The asynchronous component adds explicit state transitions, loading feedback, safe response rendering, accessible error recovery, request identity, and lifecycle cleanup.

The recorded automated tests provide evidence for the factory, renderer, and asynchronous component contracts. Browser screenshots and DevTools inspection should be included to complete the visual and integration evidence.